

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	GAUTHAM.G	Reg. No.:	192524371
Section / Batch:	SLOT - B	Date:	02/09/2026

Code of Conduct

I, GAUTHAM G (192524371) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Gautham.G

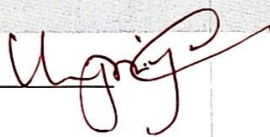
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular; well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Breadth First Search (BFS)

TO convert BFS flowchart into C program and traverse a graph using Breadth First Search.

- i) Read the number of vertices.
- ii) Read the adjacency Matrix.
- iii) Select the starting vertex.
- iv) Mark the starting vertex as visited.
- v) Insert it into the queue.
- vi) Stop when the queue become empty.

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[10][10], v[10] = {0}, q[10];
```

```
    int n, s, f = 0, r = 0, i, x;
```

```
    scanf ("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf ("%d", &a[i][j]);
```

```
    scanf ("%d", &s);
```

```
    v[s] = 1;
```

```
    q[r++] = s;
```

```
Print f ("BFS:");
```

```
while (f < r) {
```

```
    x = a[f++];
```

```
    Print f ("x: d", x);
```

```
for (i = 0; i < n; i++)
```

```
    if (a[x][i] && !v[i]) {
```

```
        v[i] = 1;
```

```
        a[r++] = i;
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

2) Kruskal's Algorithm

```
#include <stdio.h>
```

```
struct Edge {
```

```
    int u, v, w;
```

```
} e[20];
```

```
int p[10];
```

```
int find(int x) {
```

```
    while (p[x] != x)
```



```

    x = p[x];
return x;
}

int main () {
    int n, m, i, j; cost = 0, count = 0;
    scanf ("%d %d", &n, &m);
    for (i = 0; i < m; i++)
        scanf ("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
    for (i = 0; i < m - 1; i++)
        for (j = i + 1; j < m; j++)
            if (e[i].w > e[j].w) {
                t = e[i]; e[i] = e[j]; e[j] = t;
            }
    if (a != b) {
        printf ("%d - %d = %d\n", e[i].u, e[i].v, e[i].w);
        cost += e[i].w;
        p[a] = b;
        count++;
    }
    printf ("cost = %d", cost);
    return 0;
}

```

3) Topological Sort

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[10][10], in[10] = {0}, r[10], n;
```

```
    int f = 0, r = 0, i, j, x, count = 0;
```

```
    scanf ("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++) {
```

```
            scanf ("%d", &a[i][j]);
```

```
            if (a[i][j])
```

```
                in[j] ++;
```

```
    }
```

```
    for (i = 0; i < n; i++)
```

```
        if (in[i] == 0)
```

```
            r[r++] = i;
```

```
    printf ("Topological Order:");
```

```
    while (f < r) {
```

```
        x = a[f++];
```

```
        printf ("%d", x);
```

```
        count ++;
```

```
for (i = 0; i < n; i++)
```

```
if (a[x][i] && --in[i] == 0)
```

```
    a[r++] = i;
```

```
}
```

```
if (count < n)
```

```
    printf("\n cycle exists");
```

```
return 0;
```

```
}
```

4) DFS Tree

```
#include <stdio.h>
```

```
int a[10][10], v[10], n;
```

```
void DFS (int x) {
```

```
    int i;
```

```
    v[x] = 1;
```

```
    printf("v.d", x);
```

```
    for (i = 0; i < n; i++)
```

```
        if (a[x][i] && !v[i]) {
```

```
            printf("\n Tree Edge: v.d - i.d", x, i);
```

```
            DFS(i);
```

```
}
```

```
}
```

```

int main() {
    int i, j, S;
    scanf("%d", &n);
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    scanf("%d", &S);
    printf("DFS:");
    DFS(S);
    return 0;
}

```

5) Prim's Algorithm

```

#include <stdio.h>
#define INF 999
int main() {
    int a[10][10], v[10] = {0};
    int n, i, j, k, min, u, w, cost = 0;
    scanf("%d", &n);
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)

```



```
scanf("%d", &a[i][j]);
```

```
v[0] = 1;
```

```
printf("MST: \n");
```

```
for (k=0; k<n-1; k++) {
```

```
    min = INF
```

```
    for (i=0; i<n; i++)
```

```
        if (v[i])
```

```
            for (j=0; j<n; j++)
```

```
                if (!v[j] && a[i][j] &&
```

```
                    a[i][j] < min) {
```

```
                    min = a[i][j];
```

```
                    u = i;
```

```
                    w = j;
```

```
                }
```

```
printf("%d - %d = %d\n", u, w, min);
```

```
cost += min;
```

```
v[w] = 1;
```

```
}
```

```
printf("Cost = %d", cost);
```

```
return 0;
```

```
}
```